

High-Level Design (HLD)

Mutual Fund Purchase Optimizer System

1. System Overview

The Mutual Fund Purchase Optimizer is a decision-support system designed to recommend optimal mutual fund allocations based on:

- Investment amount
- Investment horizon
- Risk profile
- Existing portfolio

2. Problem Statement

Investors and advisors face difficulty in allocating a fixed investment amount across multiple mutual funds due to:

- Multiple fund choices
- Risk-return trade-offs
- Portfolio diversification requirements
- Alignment with existing holdings

This system solves the problem by providing:

- Optimized allocation
- Risk-adjusted recommendations
- Transparent explanation

3. Actors

Client (Investor)

- Inputs investment details
- Views recommendations and explanations

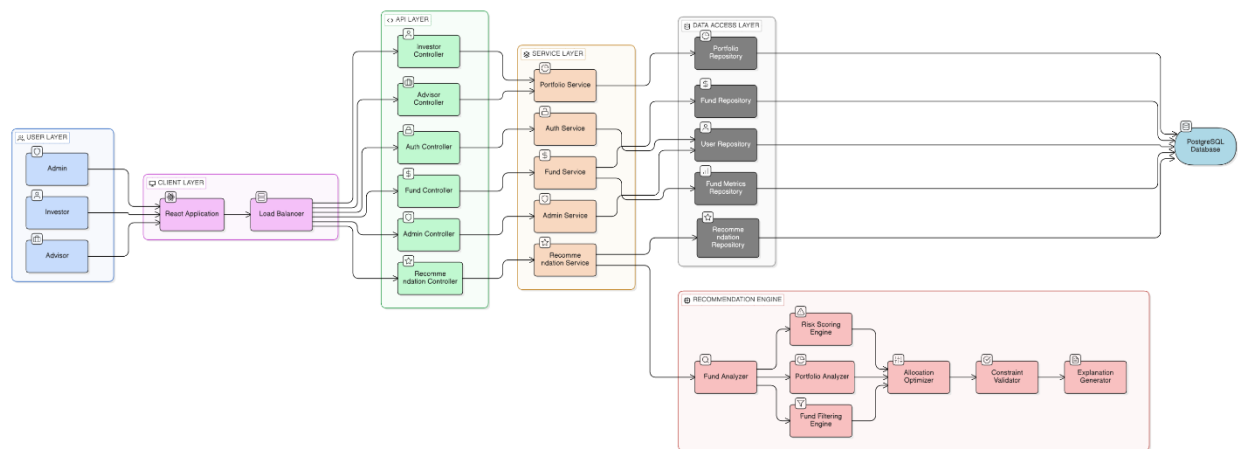
Advisor

- Reviews client portfolio
- Suggests adjustments

Admin

- Manages mutual fund data
- Configures scoring and risk rules

4. High-Level Architecture



The system follows a layered and modular architecture:

Architectural Layers

1. Presentation Layer (Frontend)
2. Load Balancer
3. API Layer
4. Backend Services Layer
5. Recommendation Engine
6. Data Layer
7. External Data Sources

5. Major System Components

5.1 Presentation Layer (Client Interface)

Technology: React

Responsibilities:

- Collect user inputs:
 - Investment amount
 - Duration
 - Risk profile
- Display:
 - Fund allocation
 - Risk alignment
 - Explanation
- Provide dashboards for:
 - Client
 - Advisor
 - Admin

5.2 Load Balancer

Example: AWS ALB

Responsibilities:

- Route incoming requests
- Distribute traffic across backend instances
- Ensure high availability

5.3 API Layer (FastAPI)

This layer acts as the entry point to backend services.

Responsibilities:

- Input validation (Pydantic schemas)
- Authentication & Authorization (JWT)
- Request routing to services
- Response formatting (JSON/REST)

Example APIs:

- /auth/login
- /portfolio/add
- /recommendation/run

5.4 Backend Services Layer

This layer contains modular business services:

Services:

- Authentication Service
- Client(Investor) Profile Service
- Portfolio Management Service
- Advisor Service
- Admin Service

Responsibilities:

- Manage users and roles
- Store and retrieve portfolio data
- Handle business workflows

5.5 Recommendation Engine (Core System)

This is the heart of the system, divided into sub-modules:

◇ 1. Portfolio Analyzer

- Analyzes existing holdings
- Detects:
 - Over-concentration
 - Asset overlap

◇ 2. Fund Filtering Engine

- Filters funds based on:
 - Category
 - Risk compatibility
 - Duration suitability

◇ 3. Risk & Scoring Module

- Computes:
 - User Risk Score
 - Fund Risk Score

- Fund Quality Score
- Uses:
 - Volatility
 - Drawdown
 - Returns
 - Sharpe Ratio

◇ 4. Constraint Validator

- Ensures:
 - Risk alignment
 - Diversification
 - Allocation limits

◇ 5. Allocation Optimizer

- Allocates investment across funds
- Objective:
 - Maximize returns
 - Minimize risk mismatch

◇ 6. Explanation Generator

- Generates human-readable insights:
 - Why fund selected
 - Risk alignment
 - Allocation logic

5.6 Data Layer

Technology: PostgreSQL

Stores:

- Mutual fund data
- NAV history
- User portfolios
- Recommendation results

Features:

- Indexed queries for performance
- Transaction management
- Data integrity constraints

5.7 Data Ingestion Module

Sources:

- Historical mutual fund data (CSV/API)

Responsibilities:

- Load NAV and fund metadata
- Precompute metrics:
 - CAGR
 - Volatility
 - Drawdown
 - Sharpe Ratio

Improves runtime performance by avoiding heavy computation during API calls.

5.8 External Data Sources

- Mutual fund datasets
- Historical NAV data

6. System Data Flow

1. Client submits investment inputs via UI
2. Request goes through Load Balancer
3. API Layer validates and authenticates request
4. Backend retrieves portfolio data
5. Portfolio Analyzer evaluates holdings
6. Risk & Scoring Module computes scores
7. Fund Filtering Engine selects eligible funds
8. Allocation Optimizer calculates allocation
9. Explanation Generator produces justification
10. Response sent back to UI

7. Optimization Flow

1. Collect user inputs
2. Compute user risk profile
3. Evaluate mutual funds:
 - Risk
 - Quality
 - Duration fit
4. Apply constraints:
 - Diversification
 - Risk mismatch
5. Rank funds
6. Generate allocation

8. Technology Stack

Layer	Technology
Frontend	React
Backend	Python, FastAPI, Uvicorn
Database	PostgreSQL
Deployment	Docker, AWS
API Format	REST, JSON

9. Non-Functional Requirements Mapping

Requirement	Architectural Support
Scalability	Stateless APIs, Load Balancer
Performance	Precomputed metrics, in-memory scoring
Security	JWT, HTTPS
Maintainability	Modular services
Availability	Health checks, container restart

10. Assumptions and Constraints

- Only historical data is used
- No real-time market integration
- Recommendations are advisory
- No trade execution

11. Future Enhancements

- ML-based recommendation engine
- Real-time market data integration
- SIP optimization
- Personalized learning system

12. Conclusion

This HLD defines a scalable, modular, and production-ready architecture for a mutual fund purchase optimizer.

The system ensures:

- Accurate recommendations
- Risk alignment
- Explainability
- High performance and scalability